MA433 Homework 3

Problem 1

Problem 3.2-2 (page 168) in your textbook: A model of an automobile suspension

system is given by:

$m\ddot{y} + ky = u$

here m is the mass, k the spring constant, u the upward force on the frame, and y the

vertical position. To conduct a durability test, we repetitively apply a force u(t) and

suddenly remove it until failure occurs. To compute the force, we can solve the

following problem. Find u(t) to move the automobile from $y(0) = 0, \dot{y}(0) = 0$ to a

final position of $y(T) = h, \dot{y}(T) = 0$ at a given final time T. Minimize the control

energy:

$$J = \frac{1}{2}\int_0^T u^2 dt$$

Problem Statement in Bolza Cost form:

minimize $J = \frac{1}{2}\int_0^T u^2 dt$

subject to: $m\ddot{y} + ky = u$ with $\begin{matrix} y(0) = 0 \\ \dot{y}(0) = 0 \\ y(T) = h \\ \dot{y}(T) = 0 \end{matrix}$

Part A

Write the state equations and the Euler-Lagrange equations then eliminate u from the state and co-state equations.

State Equations

Need differential state equations in from of n first-order differnetial equations:

$m\ddot{y} + ky = u$

let $s = \dot{y}$

then $m\dot{s} + ky = u$

resulting in two state equations of y and s in the form:

$$\dot{y} = s$$
$$\dot{s} = \frac{u - ky}{m}$$

Hamiltonian Equation

The Hamiltonian of the cost and constraints is

$$H = L + \lambda^T \overline{f} = \frac{1}{2}u^2 + [\lambda_y, \lambda_s] \begin{bmatrix} s \\ \frac{u - ky}{m} \end{bmatrix}$$

$$H = \frac{1}{2}u^2 + \lambda_y s + \lambda_s (\frac{u - ky}{m})$$

Euler-Largrange Equations

Co-state Equations:

$$\dot{\overline{\lambda}}^T = -\frac{\partial H}{\partial \overline{x}} = -\left[\frac{-k}{m}\lambda_s, \lambda_y\right]$$

Due to terminal Boundary Conditions also ($y(T) = h, s_T = \dot{y}_T = 0$ set equal to zero form):

$$\Phi_f = \phi_f + \overline{\nu}_f^T \overline{\Psi}_f = [\nu_1, \nu_2] \begin{bmatrix} y_T - h \\ s_T \end{bmatrix}$$

$$\overline{\lambda}_f^T(T) = \frac{\partial \Phi_f}{\partial x_f} = \frac{\partial}{\partial x_f}(\phi_f + \overline{\nu}_f^T \overline{\Psi}_f) = [\nu_1, \nu_2]$$

Stationary Condition:

$$\frac{\partial H}{\partial \overline{u}} = \frac{\partial H}{\partial u} = 0 = u + \frac{\lambda_s}{m}$$

In the general Hamiltonian form, neither L or f are explicit functions of time, thus for all time, t, H is some constant.

Elimination of controls from the state and co-state equations:

From the stationary condition:

$$\frac{\partial H}{\partial \overline{u}} = \frac{\partial H}{\partial u} = 0 = u + \frac{\lambda_s}{m}$$
$$u = -\frac{\lambda_s}{m}$$

This is the optimal control law, u(t) in terms of yet unknown costate variables, $\lambda(t)$.

Plugging into our state equations and co-state equations, u is eliminated resulting in:

$$\dot{y} = s$$

$$\dot{s} = \frac{u - ky}{m} = -\lambda_s - \frac{ky}{m}$$

$$\dot{\lambda}_y = -\frac{k}{m}\lambda_s$$

$$\dot{\lambda}_s = \lambda_y$$

Part B

Solve the co-states EOMs, $\dot{\lambda}$, to determine the optimal control, u, in terms of the yet unknown $\bar{\nu}$ variables.

From A:

$$u = -\frac{\lambda_s}{m}$$

$$\bar{\lambda}_f^T(T) = \frac{\partial \Phi_f}{\partial x_f} = \frac{\partial}{\partial x_f}(\phi_f + \bar{\nu}_f^T \bar{\Psi}_f) = [\nu_1, \nu_2]$$

$$\dot{\bar{\lambda}}^T = -\frac{\partial H}{\partial \bar{x}} = -\left[\frac{-k}{m}\lambda_s, \lambda_y\right]$$

Solving for u in terms of the new Lagrange multipliers by solving the coupled first order ODEs:

```
clear; clc;
syms lambda_y(t) lambda_s(t) k m nu_1 nu_2 T u omega_n
```

You may have an issue with calling this here, however MATLAB syms was introducing singularites without it:

```
assume([omega_n m k]==1);
```

dsolve():

```
f1 = diff(lambda_y) == omega_n^2*lambda_s;
f2 = diff(lambda_s) == -lambda_y;
sol = dsolve([f1, f2, lambda_y(T) == nu_1, lambda_s(T) == nu_2]);
lambda_y = simplify(sol.lambda_y,'steps',100)
```

lambda_y = $\nu_1 \cos(T - t) - \nu_2 \sin(T - t)$

```
lambda_s = simplify(sol.lambda_s,'steps',100)
```

lambda_s = $\nu_2 \cos(T - t) + \nu_1 \sin(T - t)$

```
u = -lambda_s/m;
u = simplify(u, 'Steps', 100);
disp(u);
```

$-\nu_2 \cos(T - t) - \nu_1 \sin(T - t)$

```
u = subs(u,[omega_n m],[1 1])
```

u = $-\nu_2 \cos(T-t) - \nu_1 \sin(T-t)$

Part C

Plug u(t) into the 2nd order ODE and solve for y(t). Remember that y(t) has

a particular (Hint: The system is being excited at resonance so you need to

use a modified version of the common particular solution) and a

homogeneous solution.

I did this multiple times in MATLAB and once by hand, determined fastest way to get this in MATLAB was dsolve():

```
syms y(t) A B C D h
ode = m*diff(y,t,2)+k*y == u;
ydot(t) = diff(y,t);
ySol = dsolve(ode,[y(T) == h,ydot(T)==0]);
y = simplify(ySol,'Steps',100)
```

y = $h\cos(T-t) - 0.5000\,\nu_1 \sin(T-t) + 0.5000\,T\,\nu_1 \cos(T-t) - 0.5000\,T\,\nu_2 \sin(T-t) - 0.5000\,\nu_1\,t\cos(T-$

Part D

Use the boundary conditions to find the□ variables and thus y(t). Let m = k = 1.

```
y = subs(y,[m,k,omega_n],[1,1,1])
```

y = $h\cos(T-t) - 0.5000\,\nu_1 \sin(T-t) + 0.5000\,T\,\nu_1 \cos(T-t) - 0.5000\,T\,\nu_2 \sin(T-t) - 0.5000\,\nu_1\,t\cos(T-$

```
BC1 = subs(y, t, 0) == 0
```

BC1 = $h\cos(T) - 0.5000\,\nu_1 \sin(T) + 0.5000\,T\,\nu_1 \cos(T) - 0.5000\,T\,\nu_2 \sin(T) = 0$

```
BC2 = subs(diff(y), t, 0) == 0
```

BC2 = $h\sin(T) + 0.5000\,\nu_2 \sin(T) + 0.5000\,T\,\nu_2 \cos(T) + 0.5000\,T\,\nu_1 \sin(T) = 0$

```
BC3 = subs(y, t, T) == h;
BC4 = subs(diff(y), t, T) == 0;

[nu_1_sol, nu_2_sol] = solve([BC1, BC2], [nu_1, nu_2])
```

nu_1_sol =

$$-\frac{2\,h\,(T\cos(T)^2 + \cos(T)\sin(T) + T\sin(T)^2)}{T^2\cos(T)^2 + T^2\sin(T)^2 - \sin(T)^2}$$

nu_2_sol =

$$\frac{2\,h\sin(T)^2}{T^2\cos(T)^2 + T^2\sin(T)^2 - \sin(T)^2}$$

```
y = simplify(subs(y,[nu_1 nu_2],[nu_1_sol nu_2_sol]),'steps',100)
```

```
y =
```

$$y = -\frac{\sin(T)\ (h\ (\sin(t) - t\cos(t)) - T\ h\ t\sin(t)) + T\ h\cos(T)\ (\sin(t) - t\cos(t))}{T^2 + \cos(T)^2 - 1}$$

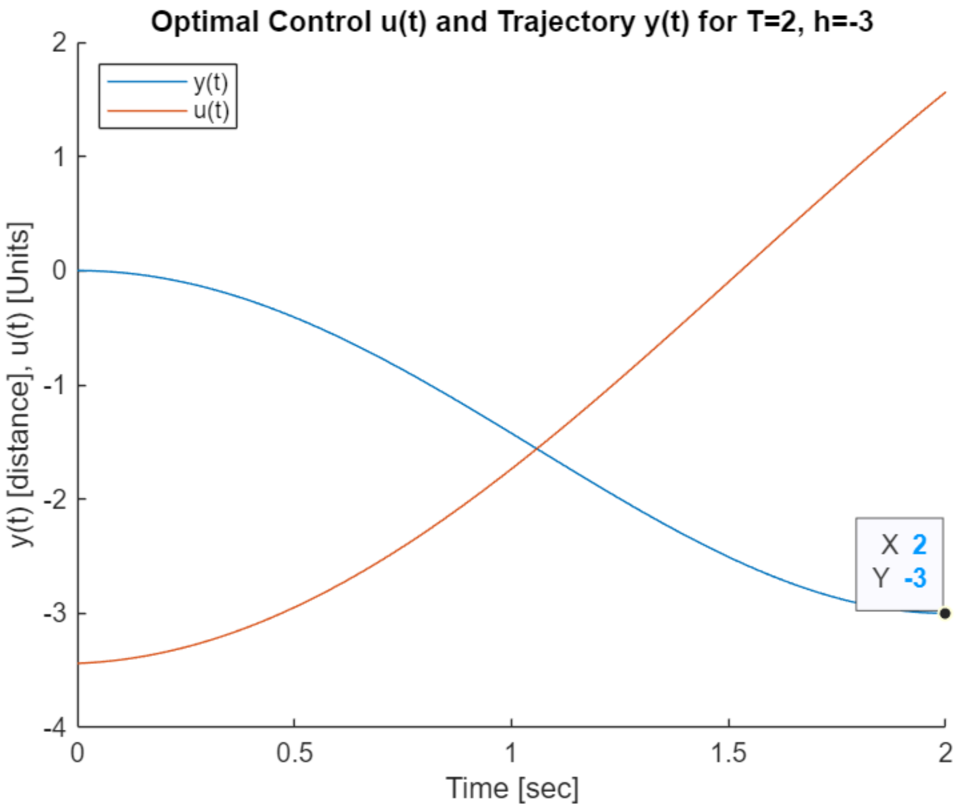
Part E

Let T = 2, h = -3. Find the optimal control and optimal state trajectory.

Plot the trajectory, y(t), and the optimal control, u(t), of the analytical

solution. Verify that y(T) = h.

```
h_val = -3;
t_f = 2;
time = linspace(0,t_f,101);
y = subs(y,[T,h],[t_f,-3]);
y = double(subs(y,t,time))';
u = subs(u,[nu_1 nu_2],[nu_1_sol nu_2_sol]);
u_sym = u;
u = subs(u,[T h],[t_f h_val]);
u = double(subs(u,t,time))';
figure();
hold on
plot(time,y)
plot(time,u)
title("Optimal Control u(t) and Trajectory y(t) for T=2, h=-3")
xlabel('Time [sec]')
ylabel('y(t) [distance], u(t) [Units]')
legend('y(t)','u(t)','Location','northwest')
ytLine = findobj(gcf, "DisplayName", "y(t)");
datatip(ytLine,2,-3);
hold off
```



To confirm target reached:

```
fprintf("y(t_f) = %.10f",y(length(y)))
```

```
y(t_f) = -3.0000000000
```

or see the datapoint.

Part F

Guess for BVP4C Solution

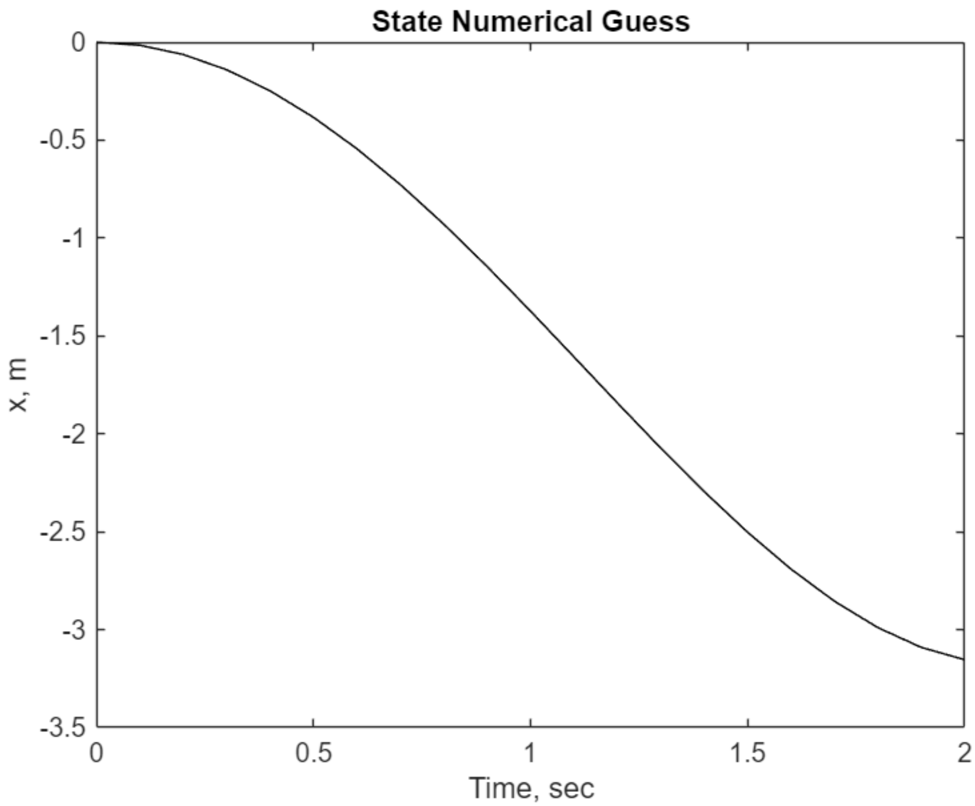
```
subs(u_sym,[T h],[t_f h_val])
```

ans = $1.5634\cos(t-2)+3.0662\sin(t-2)$

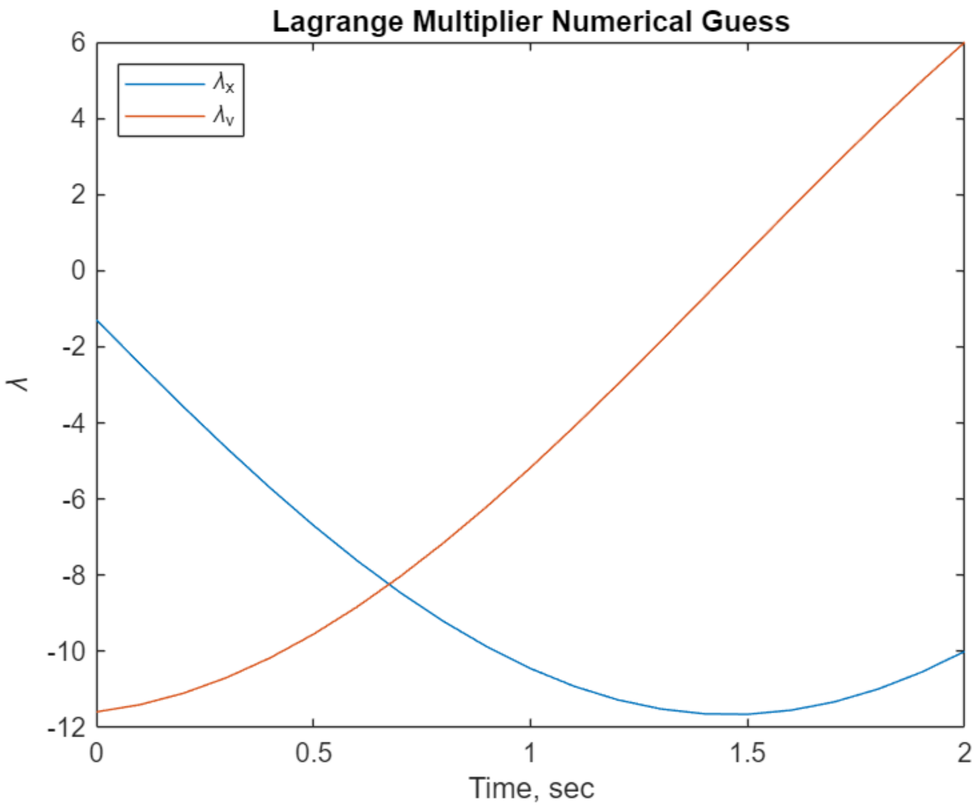
```
t_f = 2;
```

Guess for bvp4c

```
[T,S] = ode45(@(t,s)[s(2);((1.*cos(t-2)+3.*sin(t-2))-s(1))],linspace(0,t_f,21),
[0,0]);
plot(T,S(:,1),'-k')
title('State Numerical Guess')
xlabel('Time, sec')
ylabel('x, m')
```

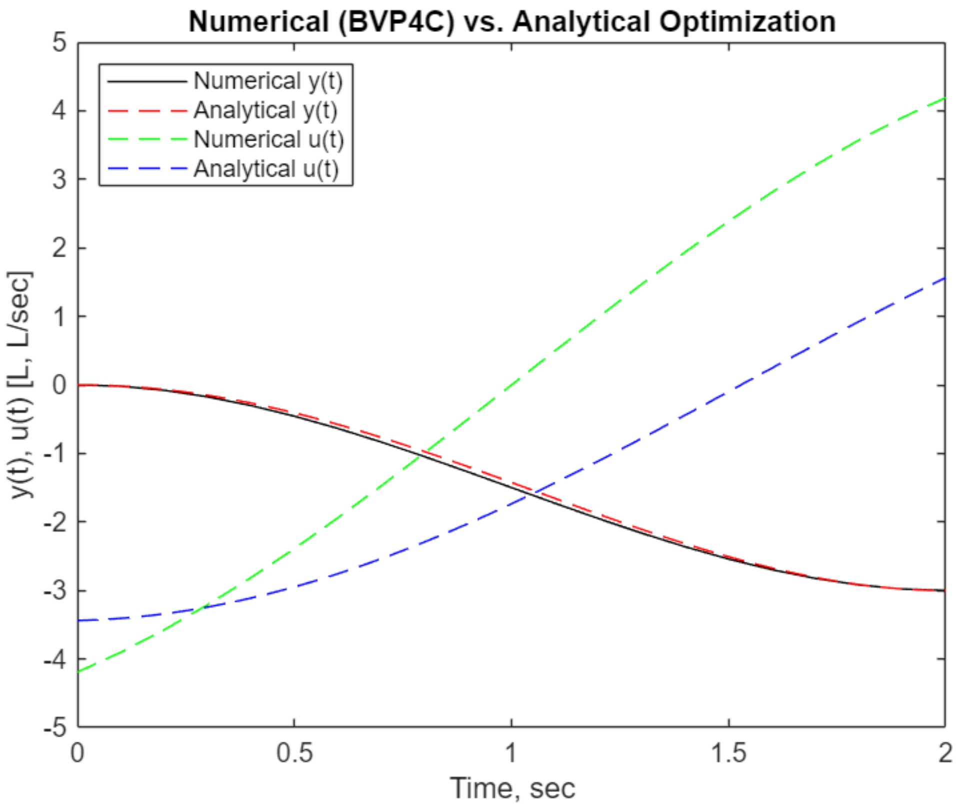



```
[T1,Lambda] = ode45(@(t,s)[s(2);-s(1)],T(end:-1:1),[-10,6]);
figure
plot(T1,Lambda)
title('Lagrange Multiplier Numerical Guess')
xlabel('Time, sec')
ylabel('\lambda')
legend('\lambda_x','\lambda_v','Location','northwest')
```



BVP4C initial structure and solution

```
solinit.x = T;
solinit.y = [S,Lambda(end:-1:1,:)]';
x_f = -3;
sol = bvp4c(@ELEOMS,@(so,sf)bcfunc(so,sf,-3),solinit);
figure();
plot(sol.x,sol.y(1,:),'-k',time,y,'--r',sol.x,-sol.y(4,:),'--g',time,u,'--b')
xlabel('Time, sec')
ylabel('y(t), u(t) [L, L/sec]')
title('Numerical (BVP4C) vs. Analytical Optimization')
legend('Numerical y(t)','Analytical y(t)','Numerical u(t)','Analytical
u(t)','Location','northwest')
```



Difference in control authority is interesting, however the targets and y(t) that result from both are nearly identical.

Subfunctions

Euler-Lagrange ODEs for bvp4c

```
function dS = ELEOMS(~,s)
%s(1) = y s(2) = s s(3) = lambda y s(4) = lambda_s
dS(1,1) = s(2);
dS(2,1) = -s(4);
dS(3,1) = s(4); %dlambda_ydt
dS(4,1) = -s(3);%dlambda_sdt
end
```

Boundary Condition fucntion used by bvp4c

```
function con = bcfunc(s_0,s_f,x_f)
con(1) = s_0(1);
con(2) = s_0(2);
con(3) = s_f(1)-x_f;
con(4) = s_f(2);
end
```

Problem 2

Performance index:

$$J = \frac{1}{2}x_f^T x_f + \frac{1}{2}\int_o^T (x^T x + ru^2)dt$$

Subject to: $\dot{x}_1 = x_2$
$\dot{x}_2 = u$

Part A

Find the Matrix Riccati Equation. Write it as 3 scalar differential equations with terminal boundary conditions.

Find the optimal feedback gain in terms of the scalar components of S(t).

Matrix Ricatti Equation (MRE) with terminal constraint

Using general form for Linear Quadratic Regulator (LQR),

$$J = \frac{1}{2}\bar{x}^T(T)S(T)\bar{x}(T) + \frac{1}{2}\int_0^{t_f} \bar{x}^T Q\bar{x} + \bar{u}^T R\bar{u}dt$$

from the given performance index, $S(t_f)$, Q, and R matrices are determined

$$S(t_f) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$
$$Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$
$$R = r$$

And from the differential state constraint equations, A and B are determined

$$\dot{\bar{x}} = A\bar{x} + B\bar{u}$$
$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = A\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + Bu$$
$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$
$$B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Terminal BC's provide

thus the MRE is

$$\dot{S} + SA + A^T S - SBR^{-1}B^T S + Q = 0$$

$$\dot{S} + S\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} S - S\begin{bmatrix} 0 \\ 1 \end{bmatrix}\frac{1}{r}[0 \quad 1]S + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 0$$

$$\dot{S} + S\left(\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 1 \end{bmatrix}\frac{1}{r}[0 \quad 1]\right) + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 0$$

$$\dot{S} + S\left(\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix} - \begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{r} \end{bmatrix}\right) + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 0$$

$$\dot{S} + S\begin{bmatrix} 0 & 1 \\ 1 & -\frac{1}{r} \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 0, S(t_f) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Writing this as three scalar differential equations (to be used in the integration) with assumption of terminal BCs:

Assume terminal conditions:

$$x_1(t_f) = 5$$
$$x_2(t_f) = 0$$

In Bolza cost form using an addional Lagrange multipier:

$$\Psi = V_f^T \bar{x}_f - \bar{c} = [1 \quad 0]\bar{x}(t_f) - 5 = 0$$

$$V(t_f) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Three vector differential equations:

$$\dot{S} + S\begin{bmatrix} 0 & 1 \\ 1 & -\frac{1}{r} \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = 0, S(t_f) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\dot{V} = (SBR^{-1}B^T - A^T)V, V(t_f) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\dot{P} = V^T BR^{-1}B^T V, P(t_f) = 0$$

S and $\dot{S}$ must be symmetric, thus can be written as three independent scalar differential equations:

$$\dot{S}_{1,1} = -1$$

$$\dot{S}_{1,2} = -S$$

$$\dot{S}_{2,2} = \frac{S}{r} - 1$$

The optimal control law for the MRE is defined as:

$$u(t) = -R^{-1}B^T\lambda(t) = -R^{-1}B^T\left[S\bar{x} + VP^{-1}\left(V_f^T\bar{x}_f - V^T\bar{x}\right)\right]$$

Vector optimal control law is then:

$$u(t) = -\frac{1}{r}[0 \quad 1](\begin{bmatrix} S_1 & S_2 \\ S_3 & S_4 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} V_1 & V_2 \end{bmatrix}\frac{1}{P}(\begin{bmatrix} V_1(t_f) & V_2(t_f) \end{bmatrix}\begin{bmatrix} x_1(t_f) \\ x_2(t_f) \end{bmatrix} - \begin{bmatrix} V_1 & V_2 \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

```
syms u(t) r S_1 S_2 S_3 S_4 x_1 x_2 V_1 V_2 P V_1f V_2f x_1f x_2f
u(t) = (-1/r)*[0,1]*([S_1, S_2; S_3, S_4]*[x_1;x_2]+[V_1; V_2]*(1/P)*(5-[V_1,
V_2]*[x_1;x_2]))
```

```
u(t) =
```

$$-\frac{S_3\,x_1 + S_4\,x_2 - \frac{V_2\,(V_1\,x_1 + V_2\,x_2 - 5)}{P}}{r}$$

This is the optimal feedback gain in terms of scalar components of S(t):

$$u(t) = -\frac{S_3x_1 + S_4x_2 - \frac{V_2(V_1x_1 + V_2x_2 - 5)}{P}}{r}$$

Part B

Find the analytical expressions for the Algebraic Riccati Equation and optimal gain.

Algebraic Ricatti Equation (ARE)

$$SA + A^TS - SBR^{-1}B^TS + Q = 0$$

$$S\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}S - S\begin{bmatrix} 0 \\ 1 \end{bmatrix}\frac{1}{r}[0 \quad 1]S + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \bar{0}$$

$$S\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}S - S\begin{bmatrix} 0 & 0 \\ 0 & \frac{1}{r} \end{bmatrix}S + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \bar{0}$$

$$\begin{bmatrix} 1 - \frac{S_2S_3}{r} & S_1 - \frac{S_2S_4}{r} \\ S_1 - \frac{S_3S_4}{r} & S_2 + S_3 - \frac{S_4^2}{r} + 1 \end{bmatrix} = \bar{0}$$

The optimal control law (gain) resulting from the ARE is as follows:

$$u(t) = -R^{-1}B^T\lambda(t) = -R^{-1}B^TS\bar{x}$$

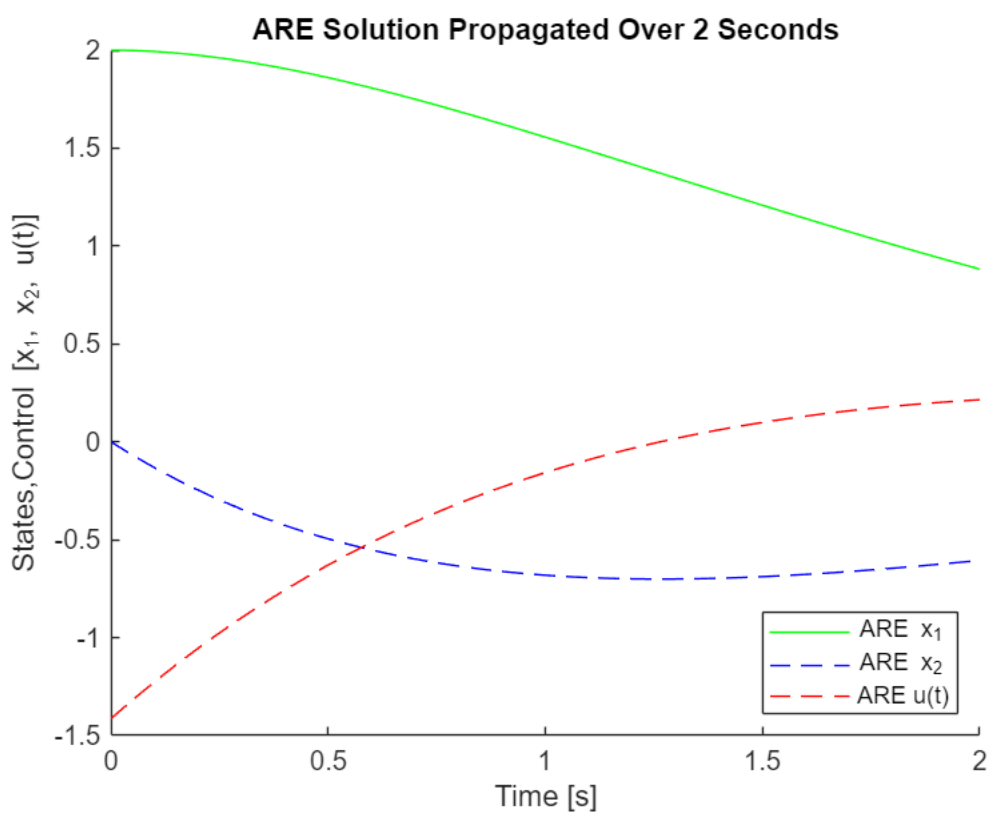
Part C

Numerically solve the problem where the system starts at rest and x1(0) =2 m. Let r = 2 and T be 2 seconds. Compare the Matrix Riccati solution (you must integrate the 3 scalar differential equations backwards from 2 seconds to 0 to determine S(t)) to the Algebraic Riccati solution (use icare in MATLAB) by plotting the states and control for the 2 solutions on the same figures. What should happen as T increases? (You can plot the 2 solutions for T = 5 seconds to see).

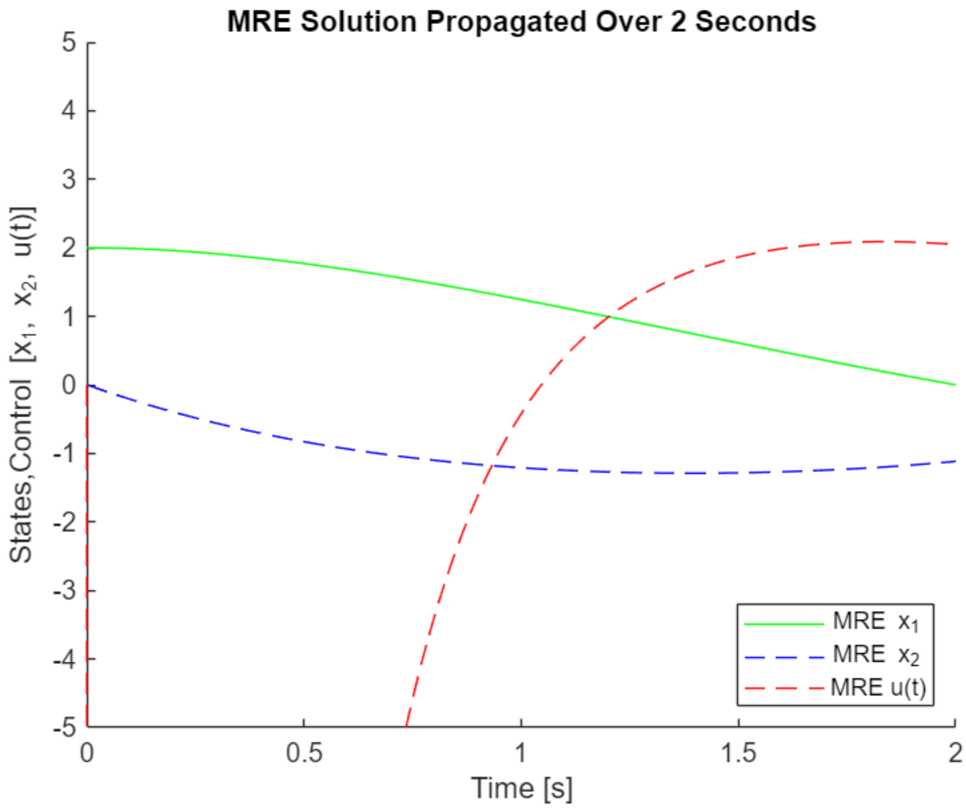
```
A = [0 1;0 0];
B = [0;1];
Q = eye(2);
R = 2;
T_f = 2;
S = icare(A,B,Q,R)
```

```
S = 2×2
        1.95663668695703          1.41421356237309
        1.41421356237309           2.7671021393314
```

```
[T,States] = ode45(@(t,s)stateEOMs(t,s,R,B,S),0:0.01:T_f,[2,0]);
figure();
hold on
plot(T,States(:,1),'g-')
plot(T,States(:,2),'b--')
ARE_u = zeros(201, 1);
for i = 1:201
    current_state = States(i, :)';
    ARE_u(i) = -(R^-1) * B' * S * current_state;
end
plot(T,ARE_u,'r--')
title("ARE Solution Propagated Over 2 Seconds")
xlabel("Time [s]")
ylabel("States,Control [x_1, x_2, u(t)]")
legend("ARE x_1","ARE x_2","ARE u(t)",'Location','southeast')
hold off
```



```
figure();
hold on
[T,MREStates] = ode45(@(t,s)MREodeCon(t,s,A,B,Q,R),T_f:-0.01:0,[1,0,0,1,1,0,0]);
[T1,States_MRE] = ode45(@(t,s)stateMREEOMs(t,s,B,R,T,MREStates),0:0.01:T_f,[2,0]);
MRE_u = zeros(201, 1);
for i = 1:201
    MRES = reshape(MREStates(i, 1:4), 2, 2);
    MREV = reshape(MREStates(i, 5:6), 2, 1);
    MREP = MREStates(i, 7);
    MREs = States_MRE(i,:)';
    MREPinv = MREP^-1;
    if MREPinv == Inf
       MREPinv = -1e1;
    end
    MRE_u(i) = -(R^-1) * B' * (MRES * MREs + MREV * (MREPinv) * (-MREV'*MREs));
end
plot(T1,States_MRE(:,1),'g-')
plot(T1,States_MRE(:,2),'b--')
xlim([0 2])
ylim([-5 5])
plot(T1,MRE_u,'r--')
xlim([0 2])
ylim([-5 5])
title("MRE Solution Propagated Over 2 Seconds")
xlabel("Time [s]")
ylabel("States,Control [x_1, x_2, u(t)]")
legend("MRE x_1","MRE x_2","MRE u(t)",'Location','southeast')
hold off
```



Plotting on top of each other beacuse its nice to compare.

Absolute error for each method:

```
fprintf("ARE Absolute Error: %.10f",States(201,1)-0)
```

```
ARE Absolute Error: 0.8818375796
```

```
fprintf("MRE Absolute Error: %.10f",States_MRE(201,1)-0)
```

```
MRE Absolute Error: 0.0000077281
```

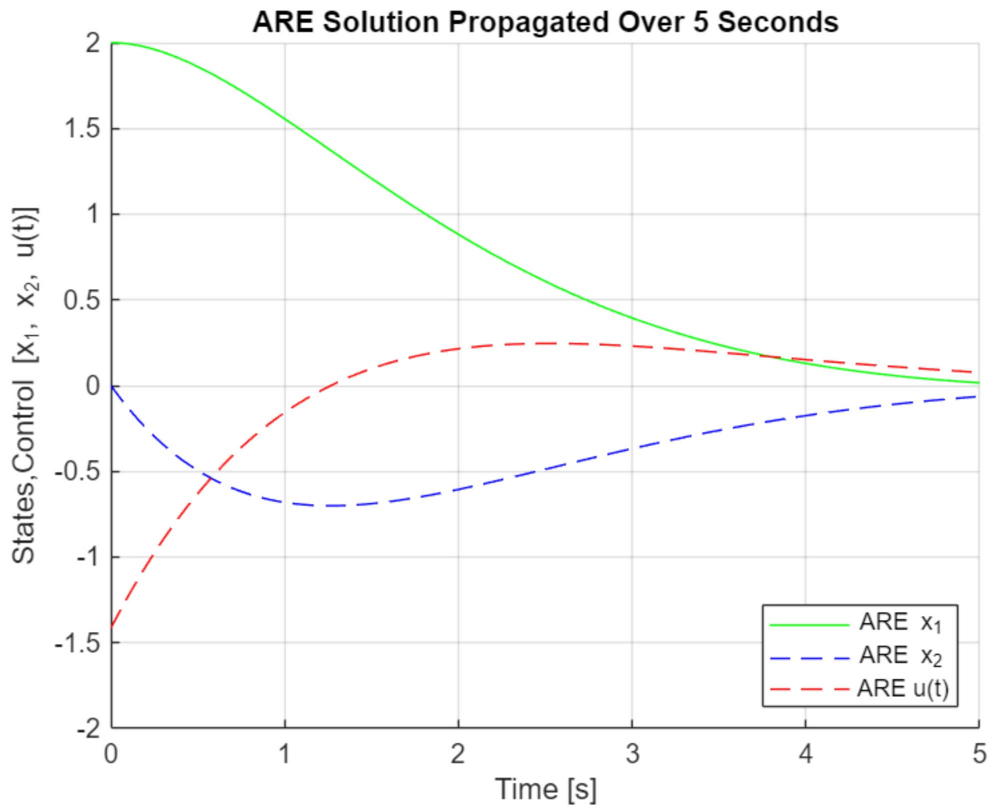
As T increases for propagation, ARE shoul dbecome more accurate (without changing Q or R as control weights

ARE is not "stiff" enough for the short time periods).

Propogation over 5 seconds to see what happens as T increases:

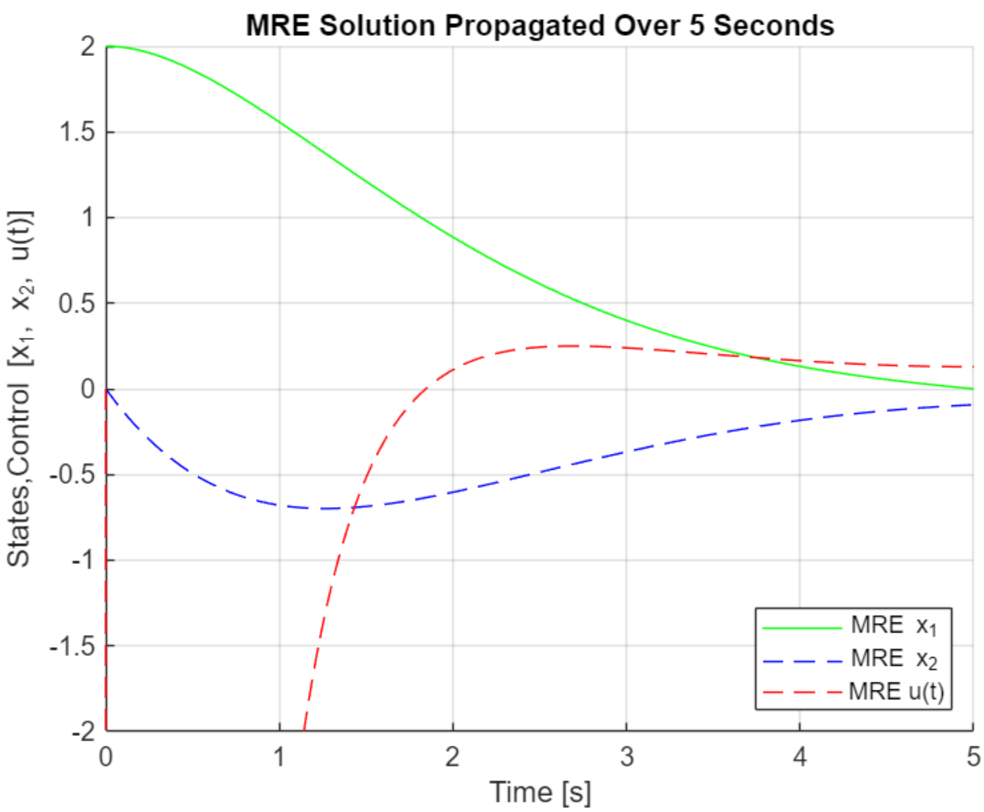
```
T_f = 5;

[T,States] = ode45(@(t,s)stateEOMs(t,s,R,B,S),0:0.01:T_f,[2,0]);

figure();
hold on
grid on
ARE_u = zeros(201, 1);
for i = 1:501
```

```
    current_state = States(i, :)';
    ARE_u(i) = -(R^-1) * B' * S * current_state;
end
plot(T,States(:,1),'g-')
plot(T,States(:,2),'b--')
plot(T,ARE_u,'r--')
title("ARE Solution Propagated Over 5 Seconds")
xlabel("Time [s]")
ylabel("States,Control [x_1, x_2, u(t)]")
legend("ARE x_1","ARE x_2","ARE u(t)",'Location','southeast')
xlim([0 5])
ylim([-2 2])
hold off
```



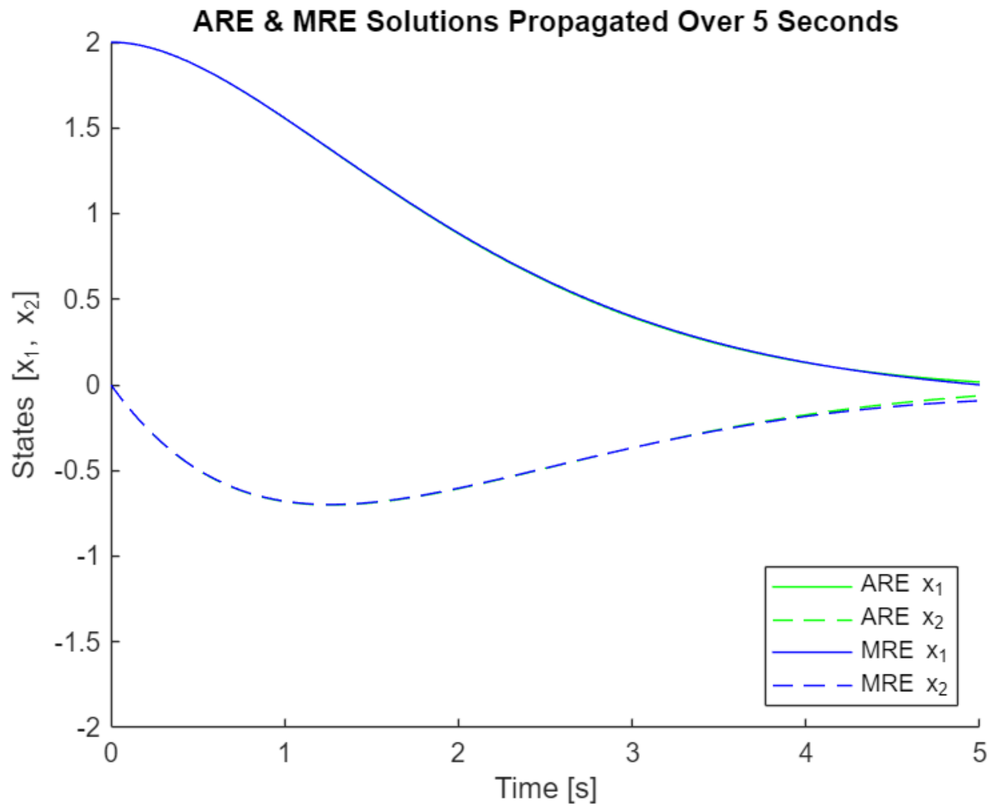

```
figure();
hold on
[T,MREStates] = ode45(@(t,s)MREodeCon(t,s,A,B,Q,R),T_f:-0.01:0,[1,0,0,1,1,0,0]);
[T1,States_MRE] = ode45(@(t,s)stateMREEOMs(t,s,B,R,T,MREStates),0:0.01:T_f,[2,0]);
plot(T1,States_MRE(:,1),'g-')
plot(T1,States_MRE(:,2),'b--')
MRE_u = zeros(201, 1);
for i = 1:501
    MRES = reshape(MREStates(i, 1:4), 2, 2);
    MREV = reshape(MREStates(i, 5:6), 2, 1);
    MREP = MREStates(i, 7);
    MREPinv = MREP^-1;
```

```
    if MREPinv == Inf
       MREPinv = -1e1;
    end
    MREs = States_MRE(i,:)';
    MRE_u(i) = -(R^-1) * B' * (MRES * MREs + MREV * (MREPinv) * (-MREV'*MREs));
end
plot(T1,MRE_u,'r--')
xlim([0 5])
ylim([-2 2])
grid on
title("MRE Solution Propagated Over 5 Seconds")
xlabel("Time [s]")
ylabel("States,Control [x_1, x_2, u(t)]")
legend("MRE x_1","MRE x_2","MRE u(t)",'Location','southeast')
hold off
```




```
figure();
hold on
plot(flip(T),States(:,1),'g-')
plot(flip(T),States(:,2),'g--')

plot(T1,States_MRE(:,1),'b-')
plot(T1,States_MRE(:,2),'b--')

title("ARE & MRE Solutions Propagated Over 5 Seconds")
xlabel("Time [s]")
ylabel("States [x_1, x_2]")
```

```
xlim([0 5])
ylim([-2 2])
legend("ARE x_1","ARE x_2","MRE x_1","MRE x_2",'Location','southeast')
hold off
```



Absolute error for each method:

```
fprintf("ARE Absolute Error: %.10f",States(501,1)-0)
```

```
ARE Absolute Error: 0.0161575087
```

```
fprintf("MRE Absolute Error: %.10f",States_MRE(501,1)-0)
```

```
MRE Absolute Error: 0.0000004215
```

As expected, as T increases, ARE becomes much more accurate and absolute errors for both methods trends down. MRE is still far superior in both cases in terms of absolute accuracy.

Subfunctions

ARE State EOMs

```
function dS = stateEOMs(~,s,R,B,S)
u = -(R^-1)*B'*S*(s);
dS(1,1) = s(2);
dS(2,1) = u;
end
```

MRE State EOMs

```
function dS = MREodeCon(~,s,A,B,Q,R)
S = reshape(s(1:4),2,2);
V = reshape(s(5:6),2,1);
Sdot = -S*A-A'*S+S*B*(R^-1)*B'*S-Q;
Vdot = (S*B*(R^-1)*B'-A')*V;
Pdot = V'*B*(R^-1)*B'*V;
dS(1:4,1) = reshape(Sdot,4,1);
dS(5:6,1) = Vdot;
dS(7,1) = Pdot;
end

function dS = stateMREEOMs(t,s,B,R,T,MREStates)
interpMRE = interp1(T,MREStates,t);
S = reshape(interpMRE(1:4),2,2);
V = reshape(interpMRE(5:6),2,1);
P = interpMRE(7);
Pinv = P^-1;
if Pinv == Inf
       Pinv = -1e1;
end
u = -(R^-1)*B'*(S*s+V*Pinv*(0-V'*s));
dS(1,1) = s(2);
dS(2,1) = u;
end
```